



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: JOSE MIGUEL CÁRCAMO GARCÍA **Carné:** 1031525

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?
 - a. Es una arquitectura de interfaz de programación de aplicaciones (API) que utiliza el protocolo HTTP para permitir la comunicación e intercambio de datos entre diferentes sistemas o aplicaciones web de forma ligera y estandarizada.
2. ¿Qué es un endpoint?
 - a. Es una URL o dirección específica expuesta por un servidor web a la que un cliente puede conectarse para acceder o manipular recursos de una API.
3. ¿Qué diferencia existe entre una URL y un endpoint?
 - a. Una URL es la dirección completa de una página o recurso en la red (ej. <https://petstore3.swagger.io/>), mientras que el endpoint se refiere a la combinación específica del camino (*path*) y método HTTP en la API para realizar una operación concreta (ej. `GET /pet/123`).
4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?
 - a. GET: Recupera o lee información/recursos del servidor sin modificar el estado.
 - b. POST: Crea un nuevo recurso en el servidor.
 - c. PUT / PATCH: Actualiza un recurso existente (PUT reemplaza el recurso completo; PATCH actualiza solo campos específicos).
 - d. DELETE: Elimina un recurso del servidor.
5. ¿Qué es JSON y por qué es común en las APIs?
 - a. JSON es un formato de texto ligero para el intercambio de datos. Es muy común porque es fácil de leer y escribir tanto para humanos como para computadoras, y es nativamente compatible con la mayoría de lenguajes de programación.
6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.
 - a. 200 OK: La solicitud se procesó con éxito.
 - b. 404 Not Found: El recurso solicitado no fue encontrado en el servidor.
 - c. 500 Internal Server Error: Ocurrió un error inesperado en el servidor al procesar la solicitud.
7. ¿Qué información se puede encontrar en la respuesta de una API?
 - a. Un código de estado HTTP, encabezados (*headers* como el tipo de contenido o fecha), y el cuerpo de la respuesta (*body*), generalmente en formato JSON con la información solicitada o con mensajes de error.
8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?
 - a. Permite verificar que los *endpoints* funcionen correctamente, validar datos de entrada/salida, automatizar pruebas, simular escenarios de error y documentar las peticiones sin necesidad de escribir código de *frontend* o *backend*.
9. ¿Qué es y para qué sirve Swagger UI?
 - a. Es una herramienta interactiva que genera documentación visual e interactiva a partir de especificaciones OpenAPI, permitiendo visualizar y probar los *endpoints* directamente desde el navegador web.
10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?
 - a. Funciona como un cliente HTTP avanzado para construir, organizar en colecciones, probar, depurar y automatizar peticiones a APIs de forma rápida e intuitiva.



1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
- Indique el método HTTP utilizado.
- Indique qué parámetro necesita la solicitud.
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
- Explique con sus palabras qué información contiene la respuesta.

Evidencia: captura de Swagger mostrando la operación y su respuesta.

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?



- ¿Qué datos fueron enviados en el body?
- ¿Qué código HTTP devolvió la API?
- ¿Cómo comprobó que el cambio realmente ocurrió?

5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
- Explique dónde se envía ese parámetro.
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:



“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.